

Athlete Platform

Raport architektury projektu Python

Zakres: analiza struktury, przepływu danych, decyzji treningowych, planowania, wykonania i testów.

Stan analizy: 29 lipca 2026. Raport dotyczy bieżącego worktree, w którym trwa migracja z wcześniejszej architektury decyzji i planera do nowszego pipeline'u.

1. Struktura projektu i granice domenowe

Obszar	Odpowiedzialność
collectors/, scripts/imports/	Import danych źródłowych: Apple Health XML i pliki FIT.
core/, schema/, repositories/	Modele dzienne, kontekst zdrowotny, DuckDB i dostęp do danych.
sleep/, health/, recovery/, performance/, engines/	Normalizacja snu, trendy zdrowotne, ocena regeneracji i obciążenia treningowego.
athlete/	Agregacja danych w AthleteState.
decision/	Jawny pipeline decyzji: diagnoza, recepta treningowa, wybór rodzaju treningu.
planner/	Katalog recept, selekcja recepty, DSL oraz kompilacja bloków treningu.
workout/, timeline/, simulator/, optimizer/	Model treningu, oś czasu, symulacja TSS, optymalizacja i eksport ZWO.
execution/	Porównanie planu z wykonaniem; obecnie kilka niespójnych implementacji.
coach/, briefing/, renderers/	Prezentacja rekomendacji i porannego briefingu.
tests/ i scripts/	Automatyczne testy oraz osobne skrypty importowe, demonstracyjne i ręczne.

Najważniejszą granicą domenową jest AthleteState: łączy zdrowie, kontekst dzienny, regenerację, wydolność oraz ostatni trening. Od niego rozpoczyna się autonomiczny pipeline decyzji.

2. Przepływ danych

Apple Health XML jest czytany strumieniowo przez AppleHealthParser, a AppleHealthImporter zapisuje surowe rekordy do tabeli health_records w DuckDB. SleepRepository i SleepBuilder budują sesje nocne, natomiast HealthRepository agreguje wybrane rekordy do HealthDaily.

Pliki FIT trafiają do FitParser, a WorkoutAnalyzer wylicza między innymi NP, IF, TSS, strefy mocy oraz metryki tętna i kadencji. WorkoutRepository zapisuje podsumowania sesji, z których WorkoutHistoryBuilder buduje historię obciążeń.

ContextBuilder wybiera ostatni dzień z danymi snu i buduje trendy HRV, RHR oraz snu dla 7 i 30 obserwacji. RecoveryEngine punktuje te trzy sygnały, a PerformanceEngine oblicza uproszczone ATL z 7 dni, CTL z 42 dni i TSB = CTL - ATL.

Apple Health / FIT	Collectors i parsers	DuckDB	Health / Recovery / Performance	AthleteState	Decision + Planner
--------------------	----------------------	--------	---------------------------------	--------------	--------------------

Uwaga: PlatformEngine nie pobiera danych samodzielnie; oczekuje już zbudowanego AthleteState.

3. Pipeline decyzyjny

1. DiagnosisEngine - interpretuje recovery.score oraz fatigue, fitness i freshness; tworzy AthleteDiagnosis z readiness, capacity, injury risk i powodami. Fatigue ≥ 80 wymusza niską gotowość i wysokie ryzyko.
2. PrescriptionEngine - mapuje diagnozę na TrainingPrescription. Wysokie ryzyko daje recovery 45 min / 30 TSS; peak prowadzi do VO2; wysoka gotowość do threshold; umiarkowana do endurance.
3. SelectionEngine - mapuje cel na WorkoutType i tworzy DecisionResult z czasem, TSS, priorytetem, intensywnością w TSS/h oraz uzasadnieniami.
4. WorkoutPlan - wybiera rekomendację o najwyższym wyniku. W aktualnym pipeline jest to na ogół pojedyncza rekomendacja, a nie konkurencja wielu niezależnych reguł.

4. Generowanie treningów

PlannerEngine realizuje łańcuch: DecisionResult -> SelectionContext -> TrainingRecipe -> DSL AST -> PlannedBlock[] -> PlannedWorkout.

- Katalog recept zawiera osiem statycznych jednostek: recovery, endurance 60/90/120, threshold 60/4x8 oraz VO2 60/Gorby.
- RecipeScorer ocenia zgodność czasu, TSS, regeneracji, zmęczenia, celu adaptacyjnego i profilu zmęczenia.
- DSL składa się z Workout, Interval i Repeat. DSLParser wybiera generator, a DSLCompiler rozwija powtórzenia do liniowej listy PlannedBlock.
- OptimizerEngine skaluje długości bloków, aby symulowany TSS odpowiadał celowi. SimulatorEngine stosuje stałe FTP = 285 W i uproszczony model TSS.
- TimelineBuilder tworzy segmenty czasowe z początkiem i końcem dla każdego bloku.

5. System wykonania treningu

W projekcie współistnieją trzy ścieżki oceny wykonania.

- ExecutionEngine: prosta, używana w scripts/execution_demo.py. Porównuje planowany i rzeczywisty czas oraz TSS; wynik to średnia dwóch score'ów, a ukończenie wymaga co najmniej 90%.
- ExecutionAnalyzer: ma wyliczać globalny wynik z mocy 50%, kadencji 20%, HR 10% i ukończenia 20%. Aktualnie jest niespójny importowo: oczekuje WorkoutExecution w execution.models, gdzie taki model nie istnieje.
- WorkoutExecution i BlockAnalyzer: analiza blokowa. Buduje timeline, dopasowuje rekordy Activity po elapsed_time, liczy średnią moc/kadencję, ukończenie oraz score bloku.

Wynik wykonania nie jest jeszcze pętlą sprzężenia zwrotnego: nie wpływa na historię, performance, diagnozę ani następną receptę treningową.

6. Testy

pytest.ini ogranicza discovery do katalogu tests/. Skrypty w scripts/tests/ są ręcznymi skryptami integracyjnymi, a nie częścią automatycznego suite'u.

Lokalizacja	Zakres	Liczba przypadków
tests/test_readiness.py	Klasyfikacja gotowości	3
tests/decision/	Scenariusz decyzji recovery	1
tests/diagnosis/	Test placeholder	1
tests/selection/	Wybór recept	8
tests/dsl/	Generatory DSL	4
tests/platform/	Scenariusze end-to-end PlatformEngine	7

Wynik uruchomienia: 24 passed in 0.08s

Brakuje konfiguracji pomiaru coverage. Automatyczne testy nie obejmują importerów, repozytoriów DuckDB, FIT parsera, recovery/performance, eksportu ZWO, optymalizatora, execution ani zgodności modeli między warstwami.

7. Ocena architektury

Mocne strony

- Czytelny, deterministyczny podział: diagnoza -> prescription -> selection -> planner.
- Modele dataclass i jawne uzasadnienia decyzji poprawiają audytowalność.
- DSL i katalog recept rozdzielają decyzję o bodźcu od struktury interwałów.
- PlatformEngine daje solidną podstawę do testów scenariuszowych end-to-end.
- Import danych jest w większości oddzielony od analizy.

Najważniejszy dług techniczny

- Migracja jest nieukończona: stare i nowe modele oraz skrypty współistnieją i część z nich nie odpowiada aktualnym sygnaturom.
- Dokumentacja opisuje SQLite, podczas gdy kod korzysta z DuckDB.
- Istnieją dwa modele treningu: Workout i PlannedWorkout, oraz różne buildery. Kontrakt między plannerem, symulatorem i execution nie jest jednolity.
- PlannerEngine nie przekazuje celu adaptacyjnego do SelectionContext, mimo że scorer potrafi go użyć.
- Czas deklarowany przez receptę, czas DSL i czas po optymalizacji mogą się rozjechać.
- ExecutionAnalyzer jest aktualnie niespójny importowo, a modele BlockExecution/WorkoutExecution występują w kilku wariantach.
- CoachEngine porównuje rekomendacje do stringów, podczas gdy nowy pipeline używa enum WorkoutType.
- HealthEngine, LoadEngine, WorkoutGenerator, RecoveryAnalyzer i część template'ów pozostają szkieletami.

Priorytety dalszego rozwoju

1. Ustalić jeden kontrakt między plannerem, modelem treningu, timeline, eksporterem i execution.
2. Dokończyć oraz podłączyć analizę wykonania do kolejnej decyzji treningowej.
3. Dodać testy integracyjne z tymczasową bazą DuckDB, parserem FIT i pełnym przepływem danych.
4. Ujednolicić konfigurację FTP, progów, okien ATL/CTL, dostępnego czasu i celów sportowych.
5. Zaktualizować dokumentację oraz skrypty do aktualnej architektury.